Total Variation Image Deblurring

An Extension of the Rudin–Osher–Fatemi Model via Convex Optimisation

Non-blind image reconstruction with synthetic degradation, isotropic TV regularisation, and a row-normalised sparse blur operator solved via CVXPY/SCS.

Reynaldi Aditama, Nam Bui, David Ou

March – April 2026

Contents

1 Introduction **3**
- 1.1 Why Bother Recovering a Blurry Image? . . . 3
- 1.2 The Rudin–Osher–Fatemi Model . . . 3
- 1.3 Our Extension: Non-Blind Deblurring with Salt-and-Pepper Noise . . . 3

2 Project Setup **4**
- 2.1 Synthetic Degradation . . . 4
- 2.2 Image Choice and Preprocessing . . . 5
  - 2.2.1 Why the Cameraman Image? . . . 5
  - 2.2.2 Why Resize to 128×128? . . . 5
  - 2.2.3 Why Normalise to $[0, 1]$? . . . 5

3 Mathematical Model At a Glance **5**
- 3.1 Complete Formulation . . . 5
- 3.2 Decision Variable . . . 5
- 3.3 Parameters . . . 6
- 3.4 Convexity . . . 6

4 Mathematical Model In Depth **6**
- 4.1 The Data Fidelity Term . . . 6
  - 4.1.1 What It Encodes: The Forward Degradation Model . . . 6
  - 4.1.2 Why the Squared L^2 Norm? . . . 7
  - 4.1.3 Why Include the Factor of $\frac{1}{2}$? . . . 7
  - 4.1.4 The Blur Operator K and Row Normalisation . . . 7
- 4.2 The Total Variation Regularisation Term . . . 8
  - 4.2.1 What Is Total Variation? . . . 8
  - 4.2.2 The Role of λ . . . 8
  - 4.2.3 Why Total Variation Over Tikhonov Regularisation? . . . 8
  - 4.2.4 Why Isotropic Over Anisotropic TV? . . . 9
- 4.3 Other Modelling Decisions . . . 9
  - 4.3.1 Why Not Add a Kernel Sparsity Term? . . . 9
  - 4.3.2 Why Not Use Blind Deconvolution to Make it More Realistic? . . . 9

5 Modelling Process **9**
- 5.1 The Degradation Pipeline in Code . . . 9
- 5.2 Building the Sparse Blur Matrix K . . . 10
- 5.3 CVXPY and the SCS Solver . . . 10

6 Results and Analysis **10**
- 6.1 Comparison Against Baselines . . . 11
- 6.2 Error Map Analysis . . . 11
- 6.3 Lambda Sensitivity Analysis . . . 12
- 6.4 Pareto Frontier: Data Fidelity vs. Total Variation . . . 12

7 Future Work and Extensions **13**
7.1 Colour Images 13
7.2 Scaling to Larger Images 13
7.3 A More Efficient Solver: The FTVd Algorithm 13

8 This Model in Context **14**
8.1 Why Industry Does Not Use This Model Directly 14
8.2 The Evolution of Image Restoration 14
8.2.1 From TV to Learned Priors 14
8.2.2 Where We Are Now: Diffusion Models 14
8.3 Why This Model Still Matters 14

1 Introduction

1.1 Why Bother Recovering a Blurry Image?

Every image captured in the real world is a corrupted version of some ideal scene. A camera slightly out of focus, a patient moving during an MRI scan, a satellite collecting photons through atmospheric turbulence — in each case the observed image $\mathbf{f}$ is the result of a clean latent image $\mathbf{u}^0$ passing through two successive degradations: a *blurring* convolution K and the addition of noise $\boldsymbol{\omega}$. Recovering $\mathbf{u}^0$ from $\mathbf{f}$ is called *image deconvolution* or, when blurring dominates, *image deblurring*.

The naive approach — computing $K^{-1}\mathbf{f}$ — fails catastrophically. Blurring operators are severely ill-conditioned: high-frequency components of $\mathbf{u}^0$ are attenuated by K, so inverting amplifies noise to the point where the result is visually meaningless. The problem is ill-posed in the sense of Hadamard: small perturbations in $\mathbf{f}$ lead to arbitrarily large perturbations in the reconstruction. The standard remedy is *regularisation* — augmenting the data-fitting objective with a penalty that encodes prior knowledge about what a plausible clean image looks like. Choosing the right regulariser is the central question in image restoration.

1.2 The Rudin–Osher–Fatemi Model

The foundational work this project builds on is the 1992 paper by Rudin, Osher, and Fatemi (ROF) [2], which proposed minimising the *total variation* (TV) of the image subject to noise-statistics constraints. The key insight of ROF is that natural images are well-described by functions of *bounded variation* — they may contain sharp edges, but their gradients are integrable. This is in stark contrast to Tikhonov-style regularisers, which penalise the squared L^2 norm of the gradient and therefore strongly prefer smooth reconstructions that wash out edges.

The continuous ROF model for pure denoising ($K = I$) is:

$$\min_u \int_\Omega |\nabla u| \, d\mathbf{x} \quad \text{subject to} \quad \int_\Omega u = \int_\Omega u_0, \quad \int_\Omega (u - u_0)^2 = \sigma^2, \tag{1}$$

where u_0 is the noisy observation and σ is the known noise standard deviation. The solution is obtained by evolving the Euler–Lagrange PDE with a gradient-projection method [2]. A common unconstrained relaxation turns the noise constraint into a Lagrangian penalty:

$$\min_u \ \frac{1}{2}\|Ku - \mathbf{f}\|_2^2 + \lambda \, \mathrm{TV}(u), \tag{2}$$

known as the TV/L^2 model [1].

1.3 Our Extension: Non-Blind Deblurring with Salt-and-Pepper Noise

We extend the ROF framework in two directions:

(i) **Non-blind deconvolution.** We include a known Gaussian blur operator K, modelling out-of-focus acquisition. "Non-blind" means K is assumed known — we are not estimating it simultaneously with the image.

(ii) **Mixed noise model.** On top of blur we add salt-and-pepper noise, modelling sensor imperfections. We deliberately keep the L^2 data term so the full problem remains convex and tractable with a general-purpose solver.

The complete discrete formulation is:

$$\min_{\mathbf{u}\in\mathbb{R}^{N^2}} \frac{1}{2}\|K\mathbf{u}-\mathbf{f}\|_2^2 + \lambda \sum_{i,j} \sqrt{(D_x\mathbf{u})_{ij}^2 + (D_y\mathbf{u})_{ij}^2} \quad \text{s.t.} \quad \mathbf{0} \le \mathbf{u} \le \mathbf{1}, \tag{3}$$

where $N = 128$, K is a $16{,}384 \times 16{,}384$ sparse row-normalised convolution matrix, and the box constraint enforces valid pixel intensities.

2 Project Setup

2.1 Synthetic Degradation

Because we work in a controlled experimental setting, we *generate* the degraded image $\mathbf{f}$ ourselves rather than using a physical photograph. This is called *synthetic degradation* and has two practical advantages: (a) we have access to the ground truth $\mathbf{u}^0$, enabling quantitative evaluation, and (b) the degradation parameters are exactly known and reproducible. Our pipeline applies two successive degradations to the clean image:

Step 1 — Gaussian blur ($\sigma = 3.0$, kernel size 11×11)

We convolve $\mathbf{u}^0$ with a Gaussian kernel $k(x, y) \propto \exp\bigl(-(x^2 + y^2)/(2\sigma^2)\bigr)$ truncated to an 11×11 support and normalised to unit sum. With $\sigma = 3.0$, the effective blur radius is substantial — the point-spread function has a standard deviation of three pixels, enough to visibly smear fine structure.

Step 2 — Salt-and-pepper noise ($p = 0.05$)

Each pixel is independently set to 0 ("pepper") with probability $p/2$ and to 1 ("salt") with probability $p/2$, leaving the remaining 95% of pixels untouched. With $p = 0.05$, roughly 1 in 20 pixels is corrupted.

After both steps the degraded image is clipped to $[0, 1]$. The resulting $\mathbf{f}$ achieves PSNR = 16.60 dB and SSIM = 0.2290 relative to the clean image, confirming substantial degradation.

2.2 Image Choice and Preprocessing

2.2.1 Why the Cameraman Image?

We use the standard `scikit-image` cameraman image, a 512×512 grayscale photograph that has been a benchmark in image processing for decades. It combines smooth regions (sky, uniform clothing), high-contrast edges (camera outline, tripod), and fine texture (distant trees). This variety means no single reconstruction method trivially succeeds on all parts of the image, making quantitative metrics genuinely informative.

2.2.2 Why Resize to 128×128?

The primary reason is computational. The CVXPY/SCS optimisation operates on a variable of size $N^2 = 16{,}384$, and the constraint matrix K is $16{,}384 \times 16{,}384$. Even though K is sparse (with roughly 1.9 million nonzeros), passing it to the solver still requires significant memory and time — the 128×128 solve already takes about 40 seconds on a standard laptop. Doubling to 256×256 would produce a $65{,}536 \times 65{,}536$ matrix and roughly a 16× increase in solver time. The Pareto and λ-sensitivity experiments are run on a further-downsampled 64×64 image since each λ value requires a fresh solve.

2.2.3 Why Normalise to $[0, 1]$?

The image is normalised via

$$\mathbf{u}^0 \leftarrow \frac{\mathbf{u}^0 - \min(\mathbf{u}^0)}{\max(\mathbf{u}^0) - \min(\mathbf{u}^0)},$$

for two reasons. First, the box constraint $\mathbf{0} \leq \mathbf{u} \leq \mathbf{1}$ is only semantically meaningful when the data occupies the same range. Second, the choice of λ has an intuitive interpretation in terms of the fidelity–variation trade-off when both are measured on pixel values in $[0, 1]$.

3 Mathematical Model At a Glance

3.1 Complete Formulation

$$\boxed{\min_{\mathbf{u} \in \mathbb{R}^{N^2}} \underbrace{\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2}_{\text{data fidelity}} + \lambda \underbrace{\sum_{i,j} \sqrt{(D_x\mathbf{u})_{ij}^2 + (D_y\mathbf{u})_{ij}^2}}_{\text{isotropic TV}} \quad \text{s.t.} \quad \mathbf{0} \leq \mathbf{u} \leq \mathbf{1}} \tag{4}$$

3.2 Decision Variable

$\mathbf{u} \in \mathbb{R}^{N^2}$ is the vectorised form of the $N \times N$ restored image ($N = 128$). The pixel at row i, column j corresponds to component $(i \cdot N + j)$ of $\mathbf{u}$.

3.3 Parameters

Symbol	Value / size	Meaning
N	128	image side length
K	$\mathbb{R}^{N^2 \times N^2}$, sparse	row-normalised Gaussian blur operator
$\mathbf{f}$	$\mathbb{R}^{N^2}$	observed (blurred + noisy) image, vectorised
λ	0.05 (main run)	regularisation strength
σ	3.0	Gaussian blur standard deviation (pixels)
p	0.05	salt-and-pepper noise probability

3.4 Convexity

The objective (4) is the sum of:

- $\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2$: a convex quadratic in $\mathbf{u}$.
- $\lambda\,\mathrm{TV}(\mathbf{u})$: a convex non-smooth function — each summand $\sqrt{a^2 + b^2}$ is jointly convex in (a, b).

The feasible set $[0, 1]^{N^2}$ is a convex polytope. The full problem is therefore a *convex programme*. CVXPY verifies this via disciplined convex programming (DCP) rules, confirming `Problem is DCP: True`.

4 Mathematical Model In Depth

4.1 The Data Fidelity Term

4.1.1 What It Encodes: The Forward Degradation Model

The data fidelity term $\frac{1}{2}\|K\mathbf{u} - \mathbf{f}\|_2^2$ measures how well the predicted degraded image $K\mathbf{u}$ matches the observation $\mathbf{f}$. It arises directly from the forward model

$$\mathbf{f} = K\mathbf{u}^0 + \boldsymbol{\omega}.$$

If $\boldsymbol{\omega} \sim \mathcal{N}(\mathbf{0}, \sigma^2 I)$ (Gaussian noise), maximum likelihood estimation gives

$$\hat{\mathbf{u}}_{\mathrm{ML}} = \arg\min_{\mathbf{u}} \ \|K\mathbf{u} - \mathbf{f}\|_2^2,$$

so the squared L^2 norm is the statistically natural fidelity term under Gaussian noise. Our noise is actually salt-and-pepper rather than Gaussian, which would suggest an L^1 data term (maximum likelihood under a Laplace model). We deliberately retain the L^2 term so that the data fidelity remains *quadratic and differentiable*, keeping the full problem in the well-studied class of convex programmes where efficient solvers exist [1]. The cost is a slight model misspecification, which shows up as residual spikes in the error map.

4.1.2 Why the Squared L^2 Norm?

There are several reasons to prefer $\|\cdot\|_2^2$ over $\|\cdot\|_2$ for the data term:

1. **Differentiability.** $\|K\mathbf{u}-\mathbf{f}\|_2^2$ is everywhere differentiable with gradient $K^T(K\mathbf{u}-\mathbf{f})$, enabling gradient-based solvers and cleaner convergence analysis.

2. **Statistical motivation.** As noted above, the squared L^2 norm arises from maximum likelihood under Gaussian noise.

3. **FFT-solvable subproblems.** In splitting-based solvers like FTVd [1], the $\mathbf{u}$-subproblem with a quadratic data term has the closed-form solution involving the 2D discrete Fourier transform:

$$\mathbf{u}^{k+1} = \mathcal{F}^{-1}\left(\frac{\mathcal{F}(D^{(1)})^* \circ \mathcal{F}(w_1^k) + \mathcal{F}(D^{(2)})^* \circ \mathcal{F}(w_2^k) + (\mu/\beta)\mathcal{F}(K)^* \circ \mathcal{F}(\mathbf{f})}{\mathcal{F}(D^{(1)})^* \circ \mathcal{F}(D^{(1)}) + \mathcal{F}(D^{(2)})^* \circ \mathcal{F}(D^{(2)}) + (\mu/\beta)\mathcal{F}(K)^* \circ \mathcal{F}(K)}\right),$$

   requiring only three FFTs per iteration [1].

4.1.3 Why Include the Factor of $\frac{1}{2}$?

The $\frac{1}{2}$ pre-factor is a normalisation convention. With it, the gradient of the data term is $K^T(K\mathbf{u}-\mathbf{f})$ rather than $2K^T(K\mathbf{u}-\mathbf{f})$. This does not change the location of the minimiser but ensures the value of λ has a consistent interpretation across scalings, and gives optimality conditions a cleaner form.

4.1.4 The Blur Operator K and Row Normalisation

The blur operator $K \in \mathbb{R}^{N^2\times N^2}$ represents discrete convolution with the Gaussian kernel k_σ. For an interior pixel (i,j), the corresponding row of K contains the kernel weights:

$$(K\mathbf{u})_{ij} = \sum_{p,q} k_\sigma(p,q)\, u_{i+p-r,\, j+q-r}, \quad r = \lfloor \text{kernel size}/2 \rfloor.$$

For *boundary pixels*, the kernel support partially lies outside the image domain. Under zero-padding, the missing pixels contribute zero, so the kernel weights within the domain sum to less than 1. This makes the forward model inconsistent at boundaries. To correct this, we apply *row normalisation*:

$$K_{\text{norm}} = D^{-1}K, \qquad D_{ii} = \sum_j K_{ij}.$$

After normalisation every row sums to exactly 1 (verified numerically). The final matrix has shape $16{,}384 \times 16{,}384$ with approximately 1.9 million nonzeros (density $\approx 0.71\%$).

4.2 The Total Variation Regularisation Term

4.2.1 What Is Total Variation?

For a continuous function $u : \Omega \to \mathbb{R}$, the total variation is

$$\mathrm{TV}(u) = \int_\Omega |\nabla u| \, d\mathbf{x} = \int_\Omega \sqrt{u_x^2 + u_y^2} \, dx \, dy.$$

It measures the total "amount of oscillation" in u. For a discrete $N \times N$ image with finite differences D_x, D_y, the *isotropic* discrete TV is:

$$\mathrm{TV}_{\mathrm{iso}}(\mathbf{u}) = \sum_{i=1}^{N} \sum_{j=1}^{N} \sqrt{(D_x \mathbf{u})_{ij}^2 + (D_y \mathbf{u})_{ij}^2}, \tag{5}$$

where $(D_x \mathbf{u})_{ij} = u_{i,j+1} - u_{i,j}$ and $(D_y \mathbf{u})_{ij} = u_{i+1,j} - u_{i,j}$.

Why is TV the right regulariser for images? The key result is that minimising TV subject to data constraints yields *piecewise-smooth* solutions that respect sharp discontinuities, as the solution lives in the space $\mathrm{BV}(\Omega)$ of functions of bounded variation, which allows jump discontinuities (edges) while remaining well-defined [2]. Tikhonov regularisation $\|\nabla u\|_2^2$ penalises jumps so heavily that the minimiser is necessarily smooth and cannot reproduce step edges.

4.2.2 The Role of λ

The parameter $\lambda > 0$ controls the fundamental trade-off in (4):

- **Small** λ: the data term dominates, so the solver fits $\mathbf{f}$ closely. Since $\mathbf{f}$ contains noise, the noise is reproduced in $\mathbf{u}$.
- **Large** λ: the TV term dominates, heavily penalising spatial variation. The result becomes over-smoothed and loses fine detail.

The optimal λ lies between these extremes. In our Pareto frontier experiment (Section 6.4), the best PSNR is achieved at $\lambda = 0.02$ on the 64×64 image.

4.2.3 Why Total Variation Over Tikhonov Regularisation?

Tikhonov regularisation [1] penalises $\sum_j \|D^{(j)} u\|^2$, giving a quadratic objective that is easy to minimise by solving a linear system. The drawback is that the minimiser is necessarily smooth: any function minimising $\|\nabla u\|_2^2$ will have small gradient everywhere, blurring edges. As Wang et al. note, Tikhonov-like regularisers "tend to make images overly smooth, often failing to adequately preserve important image attributes such as sharp edges" [1].

TV regularisation is *edge-preserving*: the subdifferential of $|v|$ at $v = 0$ is $[-1, 1]$, so TV does not penalise a constant region at all but linearly penalises the magnitude of jumps. The solution can be piecewise-constant with sharp transitions, matching the BV structure of natural images.

4.2.4 Why Isotropic Over Anisotropic TV?

The isotropic form (5) uses the ℓ^2 norm of each gradient vector. There is also an *anisotropic* version:

$$\mathrm{TV}_{\mathrm{aniso}}(\mathbf{u}) = \sum_{i,j} \big(|(D_x \mathbf{u})_{ij}| + |(D_y \mathbf{u})_{ij}| \big).$$

The critical difference is *rotational invariance*. Isotropic TV penalises a diagonal edge exactly the same as a horizontal edge of equal gradient magnitude. Anisotropic TV has preferred axes: a diagonal edge incurs $\sqrt{2}$ times more penalty than a horizontal one of equal magnitude, creating visible staircase artifacts where the minimiser approximates diagonal transitions with horizontal/vertical jumps.

We use isotropic TV (implemented via `cp.tv(u_2d)` in CVXPY). This is the standard choice for photographic images where edges can appear in any orientation. Wang et al. show that extending half-quadratic methods to the isotropic case is non-trivial but enables better image quality, particularly for colour images [1].

4.3 Other Modelling Decisions

4.3.1 Why Not Add a Kernel Sparsity Term?

In *blind* deconvolution, both the image $\mathbf{u}$ and the kernel k_σ are unknown and must be estimated simultaneously. A common kernel regulariser is a sparsity penalty to prevent the trivial solution $\mathbf{k} = \delta$. In our setting, K is *known*, so no kernel regularisation is needed. Moreover, including a kernel term would break convexity in $(\mathbf{u}, K)$ — the bilinear product $K\mathbf{u}$ is not jointly convex. Keeping K fixed preserves convexity in $\mathbf{u}$ alone.

4.3.2 Why Not Use Blind Deconvolution to Make it More Realistic?

Blind deconvolution is a far harder non-convex problem, typically solved with alternating minimisation that only guarantees convergence to a local minimum. It also requires additional priors on the kernel structure. Our focus is on the mathematical properties of TV regularisation and the convex optimisation framework — keeping K known isolates these effects cleanly and preserves the guarantees of convex optimisation.

5 Modelling Process

5.1 The Degradation Pipeline in Code

The notebook implements the following sequence:

1. Load the cameraman image via `skimage.data.camera()`, resize to 128×128 with anti-aliased bilinear interpolation, and normalise to $[0, 1]$.

2. Generate the Gaussian kernel: a δ-impulse filtered with `scipy.ndimage.gaussian_filter` at $\sigma = 3$, then normalised to sum to 1.

3. Apply the blur via `gaussian_filter(u_true, sigma=3)`.
4. Add salt-and-pepper noise: draw a uniform random matrix; set pixels to 0 where the draw falls below $p/2$ and to 1 where it exceeds $1 - p/2$.
5. Clip to $[0, 1]$.

5.2 Building the Sparse Blur Matrix K

Although Gaussian blur is most efficiently applied as a 2D convolution using `gaussian_filter`, CVXPY requires an explicit matrix representation because it reasons about the problem symbolically. We build $K \in \mathbb{R}^{N^2 \times N^2}$ as a sparse LIL matrix and convert to CSR format for efficient matrix–vector products.

For each output pixel (i, j) and each kernel position (k_i, k_j):

$$K[\, iN + j,\ (i + k_i - r)N + (j + k_j - r)\,] \mathrel{+}= k_\sigma(k_i, k_j)$$

whenever the source pixel is within bounds. Each row is then divided by its sum (row normalisation). Construction takes approximately 3 seconds; the final matrix has 1,898,884 nonzeros (density $\approx 0.71\%$).

5.3 CVXPY and the SCS Solver

CVXPY (v1.8.2) is a Python-embedded DSL for convex optimisation. The user expresses the problem in natural mathematical notation; CVXPY converts it to a standard-form cone programme via DCP rules and passes it to a backend solver.

We use the **SCS** (splitting conic solver) backend, which solves cone programmes via ADMM. SCS is designed for large-scale problems and handles the combination of a quadratic data term and a non-smooth TV term efficiently.

The CVXPY problem definition is:

```
u     = cp.Variable(N * N)
u_2d  = cp.reshape(u, (N, N), order='C')
obj   = 0.5 * cp.sum_squares(K @ u - f_flat)
      + LAMBDA * cp.tv(u_2d)
prob  = cp.Problem(cp.Minimize(obj), [u >= 0, u <= 1])
prob.solve(solver=cp.SCS)
```

With $N = 128$, the problem has 16,384 decision variables. The solver reports status `optimal` with objective value 134.28, taking approximately 40 seconds.

6 Results and Analysis

We evaluate reconstruction quality using two standard metrics:

- **PSNR (Peak Signal-to-Noise Ratio):** PSNR $= 10\log_{10}(1/\text{MSE})$, where MSE $= \frac{1}{N^2}\|\mathbf{u}-\mathbf{u}^0\|_2^2$. Higher is better; values above 30 dB are generally considered good.
- **SSIM (Structural Similarity Index):** A perceptual metric in $[0,1]$ combining luminance, contrast, and structural comparisons. Values closer to 1 are better.

6.1 Comparison Against Baselines

We compare the TV model against three baselines: the degraded input, a Gaussian filter ($\sigma = 1.5$), and a 3×3 median filter.

Table 1: Reconstruction quality on 128×128 cameraman image.

Method	PSNR (dB)	SSIM
Degraded (blurred + S&P noise)	16.60	0.2290
Gaussian filter ($\sigma = 1.5$)	21.06	0.5278
Median filter (3×3)	22.04	0.6674
TV model (ours) $\lambda = 0.05$	**22.53**	0.6590

The TV model achieves the highest PSNR (22.53 dB), a gain of 5.93 dB over the degraded input. The SSIM of 0.6590 is slightly below the median filter (0.6674). This is unsurprising: median filtering is specifically designed for salt-and-pepper noise, replacing each pixel with the neighbourhood median which is robustly insensitive to the extreme 0/1 outliers. The TV model uses an L^2 data term that is not robust to outliers, so it cannot perfectly cancel the impulsive component, but it performs better overall because it simultaneously accounts for the blur.

6.2 Error Map Analysis

The absolute error maps $|\mathbf{u}-\mathbf{u}^0|$ reveal where each method fails. The degraded image shows large errors concentrated at S&P-corrupted pixels and diffuse blur-induced error throughout smooth regions. The Gaussian filter reduces blur-induced error but introduces a smooth haze. The median filter removes S&P spikes effectively but cannot recover fine-scale details blurred by K.

The TV model's error map is qualitatively different: residual error is concentrated mostly at edges and high-frequency textured regions (the tree line, the camera body), while smooth regions are reconstructed very accurately. This is the expected behaviour of TV regularisation — it excels at piecewise-smooth signals but smooths out fine texture, which it classifies as "unnecessary variation."

6.3 Lambda Sensitivity Analysis

Table 2: TV model metrics on 64×64 image across λ values.

λ	Fidelity $\frac{1}{2}\|Ku - f\|^2$	TV(u)	PSNR (dB)	SSIM
0.005	29.08	160.39	19.39	0.4565
0.010	29.36	120.50	20.82	0.5552
0.020	29.67	98.34	**21.28**	0.6216
0.050	30.19	81.96	21.03	**0.6266**
0.080	30.67	74.43	20.66	0.6113
0.100	30.96	71.22	20.43	0.6025
0.150	31.50	66.80	20.04	0.5856
0.200	31.95	64.21	19.79	0.5790
0.300	32.92	60.27	19.32	0.5675
0.500	35.21	54.48	18.51	0.5448

Several observations stand out. First, as λ increases, the fidelity cost rises while TV decreases, confirming the monotone trade-off predicted by Pareto frontier theory. Second, the best PSNR is achieved at the relatively small $\lambda = 0.02$, suggesting that for this noise level the data is still informative enough that strong regularisation hurts. At $\lambda = 0.5$ the image is over-smoothed and PSNR drops to 18.51 dB, barely better than the degraded input.

Third, SSIM peaks at $\lambda = 0.05$ while PSNR peaks at $\lambda = 0.02$. This discrepancy reflects the different things the two metrics measure: PSNR is a purely pixel-level L^2 distance, whereas SSIM rewards perceptual similarity. Slightly more regularisation ($\lambda = 0.05$) produces images that are perceptually closer to the ground truth even if their L^2 error is slightly larger.

6.4 Pareto Frontier: Data Fidelity vs. Total Variation

Each choice of λ in (4) selects a particular supporting hyperplane of the objective space. The collection of solutions as λ varies traces the *Pareto frontier*:

$$\mathcal{P} = \Big\{ \big(\tfrac{1}{2}\|Ku^* - f\|_2^2,\ \mathrm{TV}(u^*)\big) : u^* \text{ solves (4) for some } \lambda > 0 \Big\}.$$

Every point on $\mathcal{P}$ is Pareto-optimal: you cannot reduce one objective without increasing the other. As a supporting-hyperplane description of a convex set, the frontier is convex and monotone decreasing (lower fidelity cost, i.e. better data fit, requires higher TV, i.e. less smooth image).

From Table 2, the frontier moves from (29.08, 160.39) at $\lambda = 0.005$ to (35.21, 54.48) at $\lambda = 0.5$, sweeping a wide range. The PSNR values along the frontier are highest in the middle region around $\lambda = 0.02$, not at either extreme. This confirms the intuition that the optimal λ balances noise removal (requiring higher TV penalty) against detail

preservation (requiring lower TV penalty), and that this balance depends on the noise level relative to the image's true spatial variation.

7 Future Work and Extensions

7.1 Colour Images

The current model operates on a single-channel (grayscale) image. The natural extension to a three-channel colour image $\mathbf{u} \in \mathbb{R}^{N^2 \times 3}$ is *vectorial total variation*:

$$\mathrm{VTV}(\mathbf{u}) = \sum_{i,j} \sqrt{\sum_{c=1}^{3} (D_x \mathbf{u}^c)_{ij}^2 + (D_y \mathbf{u}^c)_{ij}^2},$$

which couples all three colour channels at each pixel, preventing colour fringing at edges. Wang et al. note that the half-quadratic FTVd framework extends naturally to colour reconstruction because the isotropic TV generalises to vector-valued images [1].

7.2 Scaling to Larger Images

Our implementation is limited to 128×128 by the CVXPY/SCS solver's complexity and the $O(N^4)$ memory required to store K in sparse format. The path to larger images lies in specialised solvers that exploit the convolution structure of K without explicitly forming the matrix. The FTVd algorithm of Wang et al. [1] achieves $O(N^2 \log N)$ per iteration (three FFTs) by working in the Fourier domain, making 1024×1024 images tractable.

7.3 A More Efficient Solver: The FTVd Algorithm

The FTVd algorithm [1] is directly applicable to our TV/L^2 model. It introduces an auxiliary variable $\mathbf{w}_i \approx D_i \mathbf{u}$ and solves the penalised problem

$$\min_{\mathbf{w},\mathbf{u}} \sum_i \|\mathbf{w}_i\|_2 + \frac{\beta}{2} \sum_i \|\mathbf{w}_i - D_i \mathbf{u}\|_2^2 + \frac{\mu}{2} \|K\mathbf{u} - \mathbf{f}\|_2^2$$

by alternating between:

- A $\mathbf{w}$-*step*: pixelwise 2D shrinkage (isotropic soft-thresholding), computable in $O(N^2)$ [1].
- A $\mathbf{u}$-*step*: a linear system solvable by three FFTs under periodic boundary conditions.

With a continuation scheme on β (starting small and doubling), the total solve requires approximately 40 FFTs and runs in under one second on a 512×512 image, compared to 40 seconds for our CVXPY/SCS approach on the 128×128 case [1].

8 This Model in Context

8.1 Why Industry Does Not Use This Model Directly

The TV/L^2 model is mathematically elegant but faces practical limitations:

1. **Computational cost.** Even with an efficient solver like FTVd, solving a convex programme at inference time is orders of magnitude slower than a forward pass through a neural network.

2. **Staircase artifacts.** TV regularisation promotes piecewise-constant solutions: smooth gradients (sky, skin tones) are approximated as flat regions separated by jumps, producing a characteristic "painted" or "cartoon" look.

3. **Parameter sensitivity.** The choice of λ is non-trivial and image-dependent. A fixed λ that works well for one image may over- or under-smooth another.

4. **Known blur assumption.** Non-blind deconvolution assumes the PSF is known. Real cameras have complex, spatially-varying PSFs that are difficult to measure.

8.2 The Evolution of Image Restoration

TV regularisation can be seen as the starting point of a long evolution in image restoration:

8.2.1 From TV to Learned Priors

The first step beyond TV was replacing the hand-designed prior with a learned one. Plug-and-play priors (PnP-ADMM) embed a trained denoiser (e.g. BM3D, NLM) into an iterative ADMM scheme, replacing the TV proximal step with a call to the denoiser. This retains the interpretability of the iterative framework while gaining the expressiveness of a data-driven prior.

8.2.2 Where We Are Now: Diffusion Models

Modern image restoration is dominated by score-based diffusion models, which learn the gradient of the log-prior $\nabla_{\mathbf{u}} \log p(\mathbf{u})$ from a large corpus of natural images. They represent the current state of the art for perceptual quality, at the cost of slow multi-step sampling (typically 100–1000 neural network evaluations per image). The connection to TV is conceptual: both define a prior on images; diffusion models simply learn a much richer, data-driven prior.

8.3 Why This Model Still Matters

Despite being superseded for consumer photography, the TV/L^2 model remains actively used in several domains:

1. **Medical imaging (MRI).** Compressed sensing MRI [1] uses TV regularisation on the sparse image gradient, enabling reconstruction from highly undersampled k-space measurements. Mathematical guarantees and interpretability matter in clinical settings.

2. **Astronomy and microscopy.** The PSF is often precisely known (e.g. from a point-source calibration), making non-blind TV deconvolution attractive for diffraction-limited imaging.

3. **Engineering inverse problems.** Electrical impedance tomography, seismic imaging, and CT reconstruction share the same mathematical structure and benefit from TV priors.

4. **Compressed sensing.** The extension noted by Wang et al. [1] replaces the blur operator K with a partial Fourier transform $K = P\mathcal{F}$, directly giving the compressed sensing reconstruction problem for signals with sparse gradients. FTVd applies with minor modifications, illustrating the cross-domain applicability of the TV/L^2 framework.

5. **Theoretical foundations.** Convex TV models are the proving ground for proximal algorithms, split-Bregman iterations, and primal–dual methods that underpin convergence analysis for modern unrolled deep networks.

References

[1] Y. Wang, J. Yang, W. Yin, and Y. Zhang, "A New Alternating Minimization Algorithm for Total Variation Image Reconstruction," *SIAM Journal on Imaging Sciences*, vol. 1, no. 3, pp. 248–272, 2008.

[2] L. I. Rudin, S. Osher, and E. Fatemi, "Nonlinear total variation based noise removal algorithms," *Physica D: Nonlinear Phenomena*, vol. 60, no. 1–4, pp. 259–268, 1992.